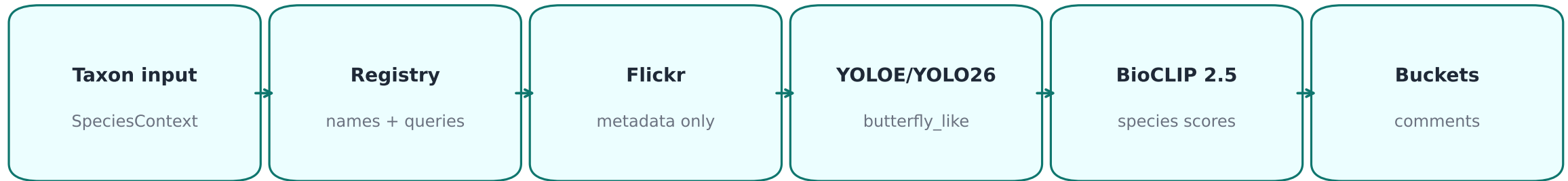


- Purpose: start from a taxon or species name, build registry-grounded Flickr discovery, detect butterfly-like objects, score them with BioCLIP 2.5, and route only ambiguous records to comment review.
- Core identity rule: GBIF accepted taxon keys and the reviewed registry define taxonomic identity; BioCLIP and comments are screening evidence.
- Durable production state is split: S3-compatible object storage holds artifacts; Postgres holds operational state and shard inventory.



S3: artifacts, raw JSON, Parquet shards, manifests, reports

Postgres: work queue, run state, completed keys, shard inventory, compaction lineage

Local/ephemeral: temporary images/crops, model runtime cache, explicit dev/test overrides

- Species/taxon input resolves to a registry-backed taxon scope and SpeciesContext.
- Registry names compile into atomic Flickr tag/text query definitions.
- Flickr work is queued in Postgres; metadata/raw JSON/canonical evidence shards are written to S3.
- Only photos with usable image URLs enter YOLOE/YOLO26 object detection; only detected butterfly_like objects enter BioCLIP 2.5 scoring.
- Joined object evidence and photo summaries produce Gold/Silver/Bronze/Bin/InReview routing and comment-review queues.

Always S3-compatible object storage

- registry/*.parquet and manifest.json
- raw Flickr JSON audit payloads
- canonical source-record Parquet shards
- object detection Parquet shards
- BioCLIP score Parquet shards
- joined evidence, summaries, metrics, reports
- future compaction outputs and manifests

Always Postgres in production

- biominer_runs: run/stage status and config
- biominer_work_items: pending/claimed/completed/failed work
- biominer_parquet_shards: committed shard inventory
- biominer_api_call_ledger: API call accounting
- biominer_compaction_inputs: source-to-output lineage

Still local / ephemeral

- explicit dev/test local filesystem outputs
- explicit dev/test SQLite datastore
- temporary downloaded images and crops inside a worker process
- model weights/cache in the runtime environment
- current local-only comment review state until cloud comment state is implemented

- resolve_taxon_scope -> build_registry -> compile_queries -> enqueue_flickr_work -> poll_flickr
- poll_flickr -> detect_objects -> score_bioclip -> join_evidence -> summarize
- summarize -> queue_comment_review -> review_comments -> apply_comment_review
- Each stage writes metrics into the run manifest; failed stages stop later dependent stages.
- Cloud workers write immutable shards and register them in Postgres instead of appending to one shared file.

Inputs

- Configured root Papilionoidea and pinned family GBIF keys
- GBIF accepted family -> genus -> species spine
- GBIF synonyms and vernaculars
- Supplemental CoL, iNaturalist, ITIS, EOL, Wikidata, reviewed translations

Outputs in S3 registry/

- taxa.parquet
- taxon_relations.parquet
- names.parquet
- name_evidence.parquet
- source_snapshots.parquet
- flickr_query_definitions.parquet
- qa_findings.parquet
- manifest.json

taxa.parquet

- accepted_taxon_key
- scientific_name
- canonical_name
- rank
- taxonomic_status
- family_key, genus_key, species_key
- parent_key and lineage fields
- registry_version, source_version

names.parquet

- accepted_taxon_key
- name, normalized_name
- name_class:
accepted/synonym/common/translation
- language/script/region fields
- source, source_record_id
- trust_tier, precision_tier, confidence
- review_state and enabled flag

name_evidence.parquet

- source evidence IDs and URLs where available
- source payload hashes or snapshots
- provider/model/version for generated names
- back-translation/corroboation state
- QA warning/fatal provenance

- Deduplication is by taxonomic identity first: accepted GBIF key, rank, and accepted scientific name are the spine.
- Name rows are deduplicated by accepted_taxon_key + normalized term + language + source/term class while preserving separate evidence rows.
- A conflicting valid family-level matcher result is fatal; lower-trust sources never replace the GBIF accepted key.
- Fatal QA blocks promotion to registry/current; warnings persist in qa_findings.parquet and reports.
- Registry manifest records version, scope hash, git SHA, source versions, build settings, counts, QA counts, file sizes, and checksums.

SpeciesContext fields

- scientific_name, canonical_name
- accepted_taxon_key
- family, genus
- family_key, genus_key, species_key
- registry_version
- synonyms, common_names, search_terms, regions
- source_versions

Artifacts

- run manifest taxon_scope
- species/<name>/species_context.json when materialized
- species_names.parquet and species_name_evidence.parquet for species subworkflow outputs
- species_registry_refresh_report.json for missing/stale refresh results

Rows generated from SpeciesContext

- accepted scientific name
- synonyms
- common names
- regional names
- reviewed translations
- broad anchored terms using the species context, not hardcoded species strings

Query columns

- registry_version, query_definition_id
- accepted_taxon_key, accepted_scientific_name
- family_key, genus_key, species_key
- term, normalized_query_term, original_term
- search_field: tags or text
- language, script, region, bbox
- source, trust_tier, precision_tier, confidence, priority, enabled

- One normalized term per query definition; tags and text are separate definitions even for the same term.
- `query_definition_id` is deterministic so reruns enqueue the same work keys and skip duplicates in Postgres.
- Tags are scheduled before text. Broad butterfly terms are scheduled with fixed upload-date slices, not recursive probe splitting.
- Every page/split transformation must carry `registry_version`, `query_definition_id`, accepted taxon key, accepted name, `family_key`, `genus_key`, and `species_key`.
- QA rejects blank terms, duplicate definitions, invalid fields, excessive broad terms, and conflicting term-to-taxon collisions.

Postgres work item payload

- `work_key = run_id:flickr:<query_hash>`
- `run_id`
- query payload: `method`, `field`, `term`, `page`, `per_page`, `lane`, `date slice`, `bbox`
- query provenance: `query_definition_id` and `taxon keys`
- `status`, `claimed_by`, `claimed_at`, `attempt_count`, `error`

Why Postgres owns this

- Multiple workers can safely claim with transactional row locks.
- Completed keys prevent duplicate API work.
- Stale claims can be requeued after worker failure.
- Follow-up pages are appended as new work items.

S3 outputs

- raw/source=flickr/method=photos_search/run_id=.../*.json
- evidence/stage=poll_flickr/run_id=.../worker=.../batch=<work_key>.parquet
- No permanent image downloads at this phase.

Canonical source-record columns

- source, flickr_photo_id, source_record_hash
- title, description, tags, owner, license
- date_taken/upload_date where available
- latitude, longitude, geo accuracy where available
- image_url and photo_page_url
- metadata evidence flags and extracted text hints

- Deduplicate photo processing, not discovery evidence.
- Canonical key: source + flickr_photo_id. One unique Flickr photo becomes one canonical source/evidence row per shard delta.
- Duplicate query discoveries are folded into arrays: text_search_terms, tag_search_terms, all_query_terms, all_query_fields, all_query_labels.
- Taxonomic provenance arrays are also folded: query_definition_ids, discovery_accepted_taxon_keys, discovery_family_keys, discovery_genus_keys, discovery_species_keys, registry_versions.
- Counters record query_hit_count and duplicate_query_hit_count; duplicate hits are evidence, not new photos to process.

Fields added or normalized

- category/life-stage hints from title, description, tags, and later comments
- hard-negative hints: artwork, tattoo, AI/generated, logo, object/product, textile/pattern, museum specimen, not-Lepidoptera
- positive hints: adult, egg, caterpillar, larva, pupa, chrysalis
- species/common-name text evidence candidates

Integrity rule

- Metadata flags route records and support bucket decisions.
- They do not make final biological classification.
- Broad butterfly query terms cannot infer family/genus/species without stronger provenance.

Source rows consumed

- Committed poll_flickr source-record shards from Postgres shard inventory
- Rows must have source + flickr_photo_id + image_url
- Work key includes run, source, photo ID, image URL, detector backend/model/checkpoint

Postgres state

- detect_objects work items are enqueued once.
- Workers claim bounded batches.
- Failed image loads become durable detection rows, not silent drops.

Detection columns

- source, flickr_photo_id, source_record_hash
- image_url, photo_page_url
- detection_id, detector_backend, prediction_source
- detector_model_id/version/checkpoint
- bbox_xyxy, bbox_xyxyn, bbox_xywhn
- detector_label, detector_score, objectness_score
- crop_hash, crop_width, crop_height, crop_storage_policy
- detection_status, failure_reason, schema_version

S3 layout

- evidence/stage=detect_objects/run_id=.../worker=.../batch=<claim_hash>.parquet
- Shard registered in biominer_parquet_shards.
- YOLOE/YOLO26 boxes are object proposals only and are not persisted as reviewed training labels.

- Only rows with `detection_status = detected` continue.
- Only rows with `detector_label = butterfly_like` continue.
- Rows with `no_detection`, `image_load_failed`, moth/other insect/object labels, or hard-negative categories do not enter BioCLIP species scoring.
- This keeps BioCLIP 2.5 focused on plausible butterfly objects and reduces cost, memory, and false positive downstream evidence.
- The gate is operational, not taxonomic: YOLOE decides whether there is a butterfly-like object; BioCLIP scores candidate species evidence.

Work item identity

- source + flickr_photo_id + detection_id + crop_hash
- model_id + model_version + model_checkpoint
- candidate_set_id + ablation_mode
- ablation modes: whole_image, detector_crop, detector_crop_segmentation where available

Score columns added

- triage_group_top and triage_group_scores
- family_top3, family_top1, family_score/margin
- genus_top8, genus_top1, genus_score/margin
- species_top20, species_top5, species_top1
- target_species_score, target_species_rank
- geospatial_prior_score/reason
- occurrence_bin and bin_reason

- Resume key excludes mutable scores and bins; it includes the object, crop, model, checkpoint, candidate set, and mode.
- Score shards are immutable S3 Parquet objects registered in Postgres.
- Temporary downloaded images/crops are deleted after scoring unless explicit debug retention is enabled.
- Candidate taxa come from the SpeciesContext/taxon scope; missing target candidates should fail clearly or require an explicit fixture override.
- Geography is a soft prior and review router. It is not an absolute taxonomic validation rule.

Central thresholds

- `gold_species_threshold` = 0.70
- `silver_species_threshold` = 0.35
- `hard_negative_threshold` = 0.70
- `ambiguous_margin_threshold` = 0.05

Bucket meanings

- Gold: strong species score, species text/name evidence, image URL, date, geo, no hard negative
- Silver: moderate score with species and parent evidence, or Gold-strength visual evidence missing date/geo
- Bronze: plausible butterfly/life-stage records that need more evidence
- Bin: not butterfly or hard-negative material
- InReview: conflicts, ambiguity, missing evidence, or text/BioCLIP disagreement

Join keys

- source + flickr_photo_id join source records to detections
- source + flickr_photo_id + detection_id + crop_hash join detections to scores
- model/checkpoint/candidate_set/mode distinguish repeated scoring runs

Joined row contains

- Canonical Flickr metadata and query provenance
- Detection boxes, labels, crop metadata, failure status
- BioCLIP family/genus/species rankings and bucket decision
- Comment-review fields, initially empty/default
- Conflict fields: Flickr text candidate, BioCLIP candidate, tag conflict

Aggregated columns

- source, flickr_photo_id
- best_detection_id
- detection_count
- best_object_occurrence_bin
- best_object_species_top1
- best_object_score
- photo_occurrence_bin, photo_bin_reason
- all_detection_ids
- all_candidate_species

Aggregation rules

- Multiple object rows per photo collapse to one photo summary.
- Best object is selected by bucket priority and score.
- Bin/hard-negative evidence can block promotion.
- All candidate species remain visible for conflict review.

Queue columns

- source, flickr_photo_id
- review_bucket, review_priority, review_reason
- best_detection_id, detection_count
- best_object_occurrence_bin
- best_object_species_top1
- best_object_score
- all_detection_ids, all_candidate_species

Queued records

- Bronze records by default
- Species conflicts
- Missing date or missing geo
- Unknown life stage
- Low confidence or ambiguous margins
- Text/BioCLIP disagreement

Target terms

- scientific name
- synonyms
- common names
- reviewed translations
- Terms are derived from SpeciesContext, not hardcoded species constants.

Promotion rules

- Bronze -> Gold only when comments support the same species and Gold metadata/negative rules pass.
- Bronze -> Silver when species is supported but Gold date/geo requirements are incomplete.
- Otherwise retain Bronze/InReview.
- Comments can promote screening confidence; they do not replace registry identity.

- Registry manifest and QA prevent invalid taxonomic scope from entering query generation.
- Deterministic IDs make reruns idempotent: query_definition_id, work_key, detection_id, crop_hash, shard_id.
- Postgres status transitions protect resumability: pending -> claimed -> completed/failed, with stale-claim requeue.
- S3 artifacts are immutable shards; compaction creates new objects and records lineage instead of mutating inputs.
- Rows carry source, flickr_photo_id, accepted taxon keys, registry versions, model checkpoints, and schema versions for auditability.
- Operational failures are recorded as failed work or failure rows; they are not interpreted as biological negatives.

Discovery and metadata

- Query definition:
deterministic
query_definition_id
- Flickr work: run_id +
query_hash
- Photo record: source +
flickr_photo_id
- Discovery provenance: folded
arrays and hit counters

Vision and scoring

- Detection: source +
flickr_photo_id +
detector_checkpoint +
normalized box + label
- BioCLIP: source +
flickr_photo_id +
detection_id + crop_hash +
model/checkpoint +
candidate_set + mode
- Evidence join: same
source/photo/object keys

Storage/control plane

- Shard: unique URI/shard_id
- Compaction: output_shard_id
+ source_shard_id lineage
- Run: run_id +
job/stage/registry_version

- registry/current/manifest.json and registry/version=<version>/*.parquet for registry inputs.
- run_id=<run>/registry/flickr_query_definitions.parquet for scoped query definitions.
- run_id=<run>/staging/raw/source=flickr/.../*.json for raw Flickr audit payloads.
- run_id=<run>/staging/evidence/stage=poll_flickr/run_id=<run>/worker=<worker>/batch=<work>.parquet for canonical source deltas.
- run_id=<run>/staging/evidence/stage=detect_objects/.../*.parquet for detection shards.
- run_id=<run>/staging/evidence/stage=score_bioclip/.../*.parquet for BioCLIP score shards.
- run_id=<run>/staging/evidence/stage=join_evidence and stage=photo_summary for downstream immutable outputs as cloud migration completes.
- run_id=<run>/reports/*.json, *.md, and review_queue shards for metrics and review artifacts.

biominer_runs

- run_id, job_name, stage
- registry_version, status
- started_at, ended_at
- config_json, summary_json

biominer_work_items

- work_key, job_name, stage, registry_version
- status, payload_json
- claimed_by, claimed_at, completed_at
- output_uri, checksum, row_count
- attempt_count, error, created_at

biominer_parquet_shards

- shard_id, job_name, registry_version, stage, run_id
- worker_id, uri, row_count, byte_count, checksum
- metadata_json, committed_at

- Worker memory: Polars frames for the currently claimed batch and model inputs.
- Temporary image bytes and crop files needed by detector/BioCLIP runtimes; these are deleted after scoring unless debug retention is explicit.
- Model runtime assets: PyTorch/BioCLIP/YOLOE weights and caches in the worker environment, not pipeline evidence artifacts.
- Developer mode: local filesystem outputs and SQLite are available only when explicitly selected for tests or smoke checks.
- Current gap: comment review state is still local SQLite in the orchestrator; cloud comment queue/state needs Postgres/S3-backed implementation.
- Current gap: join/summarize cloud stages are being moved to shard-inventory driven outputs so they do not depend on shared final Parquet files.

- Every durable table-like artifact is Parquet in S3-compatible storage; every durable report/manifest is JSON or Markdown in S3.
- Postgres carries control-plane truth, not bulk payloads: queues, status, run metadata, shard inventory, API ledgers, and compaction lineage.
- Workers claim bounded batches, write immutable shards, register shards, and then mark work completed.
- Resume reads Postgres completed keys and committed shard inventory rather than scanning local folders.
- Compaction is explicit and append-only: it writes new compacted shards and records consumed source shards.
- Local files are either explicit dev/test outputs or ephemeral runtime scratch that is not part of durable pipeline state.

Photo/source level

- One canonical row per source + flickr_photo_id after compaction/dedupe
- Folded query provenance and registry taxon provenance arrays
- Raw Flickr JSON available for audit if retained

Object level

- Zero or more detection rows per photo
- Only butterfly_like detected rows get BioCLIP score rows
- Score rows include family/genus/species top lists, target score/rank, bucket, reason

Review level

- One photo summary row per photo
- Review queue rows for Bronze/InReview/actionable cases
- Comment-reviewed evidence rows with Gold/Silver promotions where rules pass

- Before production: storage doctor validates S3 write/read/delete and Parquet scan; datastore doctor validates Postgres schema and queue claim/complete.
- Per run: manifest records stage status, counts, outputs, storage/datastore backends, model configs, and taxon scope.
- Per worker: logs and metrics include claimed work, completed work, failed work, API calls, rows written, shard URIs, and error counters.
- Test suite uses fake clients/backends; normal tests do not require live Flickr, CUDA, BioCLIP weights, Backblaze B2, or Supabase.
- Run example: `uv run biominer run --taxon "Papilio demoleus" --rank species --registry-dir s3://.../registry/current --output-prefix s3://.../runs/papilio_demoleus --storage-backend s3 --datastore-backend postgres`

Implemented or actively wired

- Production defaults prefer s3 + postgres.
- Cloud Flickr polling claims Postgres work and writes canonical S3 shards.
- Detection planning/scoring uses shard inventory and bounded claims.
- BioCLIP scoring plans from eligible butterfly_like detection shards and writes score shards.

Remaining local/shared work to finish

- Join evidence and summarize still have compatibility paths that read/write shared final files.
- Cloud comment review queue/state is not implemented in the orchestrator yet.
- Compaction exists internally but should be orchestrated as explicit cloud jobs.
- Some local object-runner helpers still materialize frames and batch files for dev/fallback paths.

- BioMiner is designed as a registry-first, evidence-preserving discovery and triage system, not a taxonomic validation publisher.
- The durable production architecture is cloud-centric: S3 for immutable artifacts and Postgres for resumable operational control.
- The key data integrity invariant is stable identifiers plus provenance preservation: dedupe processing, never erase discovery evidence.
- The visual path is detector-first: YOLOE/YOLO26 finds butterfly-like objects; BioCLIP 2.5 scores species candidates only for those objects.
- The final outputs are canonical source records, object detections, BioCLIP scores, joined evidence, photo summaries, review queues, and comment-reviewed promotions.